Lab Assignment -20

Name : Anand

Hallticket : 2303A51090

Batch No. : 02

Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code

Lab Objectives:

- Understand how to test AI-generated code for common security vulnerabilities.
- Learn to apply secure coding principles while analyzing AI outputs.
- Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption.
- Enhance code reliability and safety by using AI for secure refactoring.

Task 1 – Password Storage Vulnerability Check

Task: Analyze an AI-generated signup script for insecure password storage.

Instructions:

- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:

- A secure registration script with hashed password storage.

```python
#generate a Python user registration program. Check whether passwords are stored in plain text. Improve the script using hashing (bcrypt or hashlib).
import bcrypt
def register_user(username, password):
    # Hash the password using bcrypt
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

    # Store the username and hashed password in a dictionary (simulating a database)
    user_data = {
        'username': username,
        'password': hashed_password
    }

    return user_data
# Example usage
username = input("Enter your username: ")
password = input("Enter your password: ")
user_info = register_user(username, password)
print("User registered successfully!")
print("Stored user data:", user_info)
# Note: In a real application, you would store the user data in a secure database rather than printing it out.
```

Output :

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_20.py"
Enter your username: anandarelli
Enter your password: hello@123
User registered successfully!
Stored user data: {'username': 'anandarelli', 'password': b'$2b$12$M1MDMdS4WuczObAeLlJjS.Xf8GlUWwwHh0kf6gzjFDTcvk7Se81yq'}
PS C:\Users\arell\Music\aiac>
```

Task 2 – Hardcoded Secrets Detection

Task: Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

```python
#generate a script that connects to an external API. Identify if API keys/passwords are written directly in code. Refactor using environment variables or c
import os
import requests
def get_api_data():
    # Load API key from environment variable
    api_key = os.getenv('API_KEY')
    if not api_key:
        raise ValueError("API_KEY environment variable not set")
    # Make a request to the external API using the API key
    url = "https://api.example.com/data"
    headers = {
        "Authorization": f"Bearer {api_key}"
    }
    response = requests.get(url, headers=headers)
    return response.json()
# Example usage
if __name__ == "__main__":
    try:
        data = get_api_data()
        print("API data retrieved successfully!")
        print(data)
    except Exception as e:
        print(f"Error: {e}")
# Note: In a real application, you would set the API_KEY environment variable securely and not hard-code it in your script.

```

Output :

```
Error: API_KEY environment variable not set
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_20.py"
Error: API_KEY environment variable not set
PS C:\Users\arell\Music\aiac>
```

Task 3 – File Upload Security Validation

Task: Test an AI-generated file upload program for unsafe file handling. Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename Handling.

Expected Output:

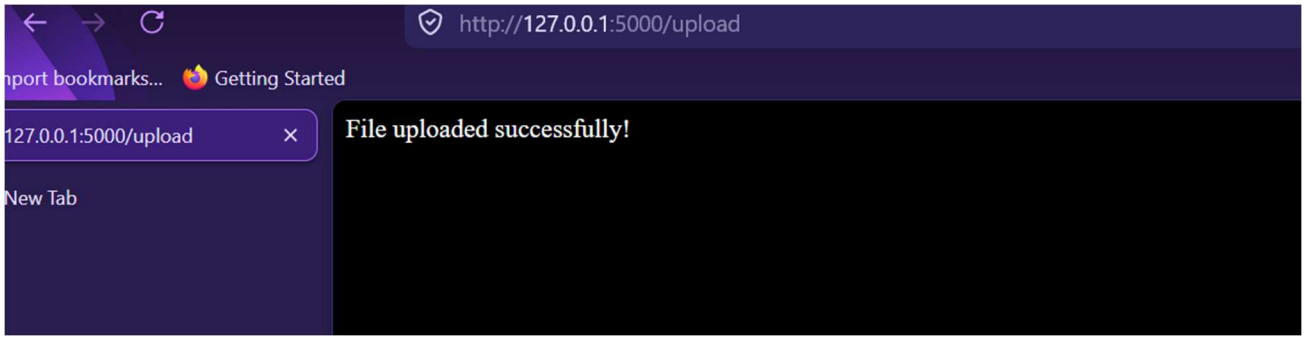
- A secure file upload implementation.

```python
#generate a Flask file upload example. Check for vulnerabilities like uploading executable files. Add validation for file type, size, and secure filename
from flask import Flask, request, redirect, url_for
from werkzeug.utils import secure_filename
import os
app = Flask(__name__)
# Configure upload folder and allowed extensions
UPLOAD_FOLDER = 'uploads/'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
def allowed_file(filename):
    return '.' in filename and \
           filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        # Check if the post request has the file part
        if 'file' not in request.files:
            return "No file part"
        file = request.files['file']
        # If user does not select file, browser also submit an empty part without filename
        if file.filename == '':
            return "No selected file"
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return "File uploaded successfully!"
        else:
            return "Invalid file type. Allowed types are: png, jpg, jpeg, gif."
    return '''
    <!doctype html>
    <title>Upload a File</title>
    <h1>Upload a File</h1>
    <form method=post enctype=multipart/form-data>
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    '''
if __name__ == '__main__':
    # Ensure the upload folder exists
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True)

```

Output :

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_20.py"
 * Serving Flask app 'assg_20'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inste
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 494-157-633
127.0.0.1 - - [04/Apr/2026 22:00:47] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 22:00:47] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 22:02:29] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 22:02:31] "GET /upload HTTP/1.1" 200 -
127.0.0.1 - - [04/Apr/2026 22:02:43] "POST /upload HTTP/1.1" 200 -
127.0.0.1 - - [04/Apr/2026 22:02:49] "GET /upload HTTP/1.1" 200 -
127.0.0.1 - - [04/Apr/2026 22:02:59] "POST /upload HTTP/1.1" 200 -
```



Task 4 – Command Injection Prevention

Task: Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using os.system() or subprocess.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

```python
#generate code using os.system() or subprocess.Identify command injection risks with user input.Fix using safe subprocess calls and input validation.
import subprocess
def run_command(user_input):
    # Validate user input to prevent command injection
    if not user_input.isalnum():
        raise ValueError("Invalid input: only alphanumeric characters are allowed.")

    # Use subprocess to run a safe command
    result = subprocess.run(
    f"echo {user_input}",
    shell=True,
    capture_output=True,
    text=True
    )
    return result.stdout.strip()
# Example usage
if __name__ == "__main__":
    user_input = input("Enter a string to echo: ")
    try:
        output = run_command(user_input)
        print("Command output:", output)
    except ValueError as e:
        print(f"Error: {e}")
```

Output :

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_20.py"
Enter a string to echo: hello
Command output: hello
```

Task 5 – Secure Input Handling in Web Forms

Task: Check AI-generated web form code for improper input validation. Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

- A secure form resistant to injection attacks.

```python
#generate an HTML + Flask feedback form.    Identify missing validation and unsafe rendering.   Fix using server-side validation and sanitization.
from flask import Flask, request, render_template_string
import html
app = Flask(__name__)
@app.route('/feedback', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        # Get user input and sanitize it
        user_feedback = request.form.get('feedback', '')
        sanitized_feedback = html.escape(user_feedback)

        # Here you would typically save the feedback to a database
        return f"Thank you for your feedback: {sanitized_feedback}"

    # Render the feedback form
    return '''
    <!doctype html>
    <title>Feedback Form</title>
    <h1>Feedback Form</h1>
    <form method=post>
      <textarea name=feedback rows=4 cols=50></textarea><br>
      <input type=submit value=Submit>
    </form>
    '''
if __name__ == '__main__':
    app.run(debug=True)

```

Output :

```
python -u "c:\Users\arell\Music\aiac\assg_20.py"
 * Serving Flask app 'assg_20'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 494-157-633
127.0.0.1 - - [04/Apr/2026 22:18:48] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 22:20:04] "GET /feedback HTTP/1.1" 200 -
```